

✓ GB3: RISC-V Processor

Department of Engineering, University of Cambridge

Author: Dr Matthew W.C. Tang (wct26@cam.ac.uk)

Acknowledgement: some materials are adopted from *Foundations of Embedded Systems - RISC-V and Open-Source FPGA Tools* from Prof Phillip Stanley-Marbell, who organised this project previously.

Introduction

Software applications reuse a small number of primitives (instructions) over and over again in time to implement a given computation. The domain of computer architecture deals with designing those primitives used by software and implementing them so that they are efficient according to some relevant measure such as time, energy, or implementation size. This IIA project will give you an opportunity to gain first-hand experience in modifying a popular computer architecture, the RV32I 32-bit integer variant of the RISC-V instruction set architecture (ISA).

The project will require you to first become familiar with the concepts of processor instructions, processor state, and processor microarchitecture. You will then explore how to implement basic hardware designs using the Verilog hardware description language and you will use this understanding to collaborate in a team to improve the performance of your microarchitectural implementation of the RV32I ISA.

RISC-V is pronounced “RISC five”.

Learning objectives

The project will introduce you to the concepts and tools for designing and modeling microprocessors, implementing hardware designs in FPGAs and evaluating their hardware resource usage, timing performance, and power dissipation.

The aim of the project is to understand how a processor is designed using Verilog, get first-hand experience implementing a baseline processor design on an FPGA, and modifying the processor to improve its hardware resource usage, timing performance, and/or power dissipation.

In this project, you will:

- learn the key concepts of computer architecture, including the concept of an instruction set architecture, processor state, and processor microarchitecture.
- learn how to use the Verilog hardware description language to design hardware.
- learn how to use open-source FPGA tools to synthesise and load Verilog hardware designs to the iCE40 FPGA.
- learn how to characterize the performance of a processor design.
- get experience optimising a processor design with respect to power consumption, timing performance, and resource usage.
- explore how to work effectively in a Team environment and identify appropriate roles for yourself

Intended Learning Outcomes

At the end of this project, you should be able to:

- create new hardware designs in the Verilog hardware description language.
- synthesise Verilog hardware designs for FPGAs using open-source FPGA tools.

- use your understanding of Verilog language and the related FPGA tools to modify a RISC-V processor design and evaluate it on an FPGA.
- propose and implement micro-architectural changes to a processor design to improve the performance and resource usage of the processor design.

Prerequisites (Assumptions)

This project assumes you to have some familiarity with the following concepts. If you are unfamiliar with any of the topics, please go through the resources recommended below.

- Basic concepts of computer architecture. If needed, you can go through the content for the Engineering Tripos Part IA (Digital Electronics and Information Processing), 3B2 (Integrated Digital Electronics) course before starting this project.
- Use of command line tools (e.g. basic shell scripts, creating Makefiles, and so on). If needed, you can watch [MIT's missing semester lecture on using the shell](#) for an introduction.

Session and Project Schedule

Compulsory sessions: Tuesdays 11-13, Fridays 9-11

Venue: EIETL (other than intro at LT6)

1. 15 May: Introduction, iCEBreaker 1 (basics)
2. 19 May: iCEBreaker 2 (Verilog testbenches)
3. 22 May: Picorv32 and PicoSoC
4. 26 May: Brainstorming
5. 29 May: Development 1
6. 2 June: Development 2
7. 5 June: Final presentations and demonstrations

Submissions

1. 21st May 4 pm (team): stopwatch.v, stopwatch_tb.v, an Excel file of data collected **[15 marks]**
2. 28th May 4 pm (team): plan, division of labour (roles and tasks) (2 pages A4 max.) **[15 marks]**
3. 11th June 4 pm (individual): final report **[50 marks]**; (group): all related design materials (diagrams, notes, code, lab books, etc.) **[no marks, supporting evidence for reports]**

Submit your work at [GB3 Moodle](#).

Guidelines for Final report

No more than 8 sides of A4, excluding appendices/references.

- Abstract (1 side)
- Summary of overall design decisions and outline of project management (1 side, possible team material)
- Description of individual design/code/test contribution (2-3 sides)
- Problems encountered by individual in development and their technical solutions (1 side)
- Conclusions, reflections and personal lessons learned (2 sides)

Note: design materials are submitted as a team so no need to repeat them in the individual report

Session 1: iCEBreaker 1 (Basics)

Recommended tasks to complete.

- Complete the stopwatch workshop (<https://github.com/icebreaker-fpga/icebreaker-workshop/tree/master/stopwatch>)
 - You may skip chapter 2, as tools had been installed.
- Read the reports from the toolchain and note the FPGA resources
- Explore the available resources on the FPGA and test it on board

✓ Hardware: iCEBreaker

This section covers the basic information of and set up for [iCEBreaker](#), the low cost, open-source educational field-programmable gate array (FPGA) development board.

The main motivating application of this board is for teaching the use of the open source FPGA design flow using Yosys, nextpnr, icespice, iverilog, symbiflow and others.

FPGA Development Toolchain

The OSS CAD suite toolchain is available on the Rocky Linux side of the EIETL PCs.

If you wish to install that onto your laptop, please follow this link <https://github.com/YosysHQ/oss-cad-suite-build>.

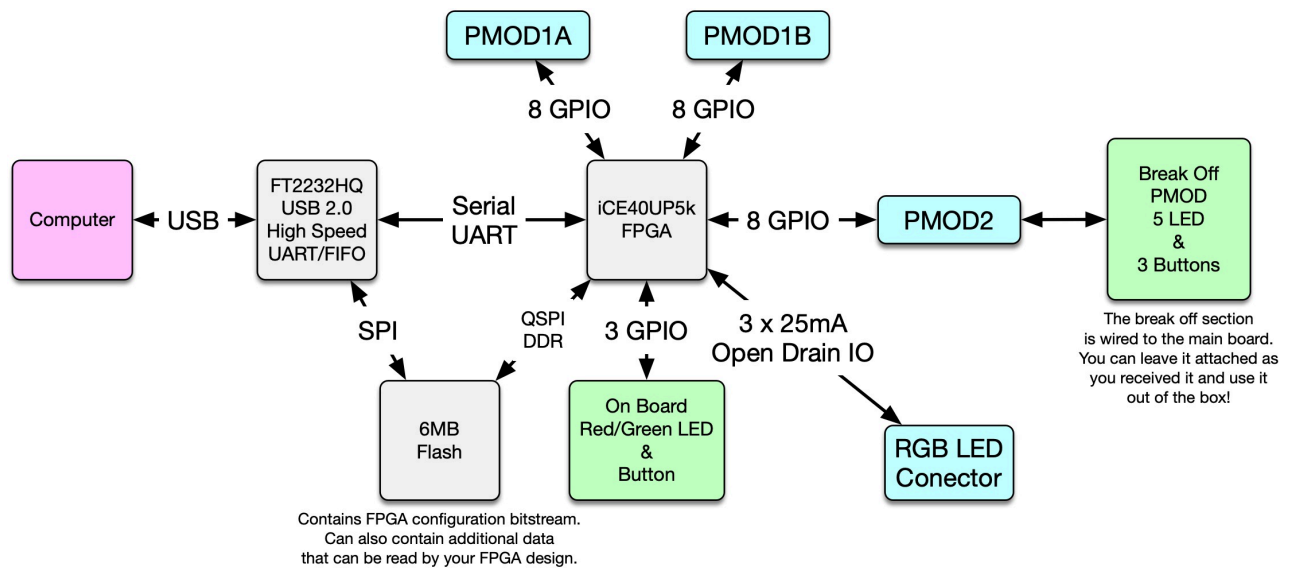
iCEBreaker Hardware Specifications

These are copied from iCEBreaker README, but only crucial resources for this project are included.

- iCE40UP5K in QFN48 (SG48) package
 - iCE40 UltraPlus 5K [Datasheet PDF Download](#)
 - **5280** Logic Cells (4-LUT + Carry + FF)
 - 128 KBit Dual-Port Block RAM
 - 1 MBit (128 KB) Single-Port RAM
 - PLL, Two SPI and two I2C hard IPs
 - Two internal oscillators (10 kHz and 48 MHz)
 - 8 DSPs (16x16 multiply + 32 bit accumulate)
 - 3x 24mA drive and 3x hard PWM IP
- QSPI-DDR-capable flash 128 MBit (16 MB)
 - The Winbond [W25Q128JVSIM](#) is used.
 - Projects may access the flash at the highest flash access speed possible.
- [FT232RL](#) interface (microUSB plug)
 - programming using [iceprog](#) or [Dimond Programmer](#)
 - 12 MHz XTAL oscillator (shared with FPGA)
- 39 I/O capable pins:
 - 4 pins for config (SDI, SDO, SCK, CSB)
 - Either loading config from on board FLASH chip or provided through the FTDI chip with direct SRAM config.
 - 2 extra GPIO pins for QSPI
 - Together with the config pins, allows storage of additional data in the FLASH chip, and high speed QSPI DDR access. For example [picoSOC](#) firmware.

- 3 PINs for RGB LED (pin header)
- 2 LEDs (one on output-only PLL pin)
- 1 Clock pin (on PLL GBIN)
- 1 UART Rx Pin via FTDI
- 1 UART Tx Pin via FTDI
- 1 Push Button
- 16 PINs on dual PMOD
- 8 PINs on single PMOD / snap-off section
- Break-off/Snap-off section (convertible to PMOD host / PMOD device)
 - 5 LEDs in similar arrangement to iCEstick
 - 3 Push Buttons
- Unpopulated parts shipped with the board
 - 3x Host PMOD (2x for dual PMOD port, 1x for snap-off section)
 - 1x Device PMOD (for other side of snap-off section)

iCEBreaker Diagrams



[illegible]

RISC-V: Picorv32 processor core

RISC-V RV32I API

- Software: RISC-V Toolchain

RISCV GNU toolchain (MacOS, Homebrew)

Add the homebrew tap from <https://github.com/riscv-software-src/homebrew-riscv> and install the tools.

```
brew tap riscv-software-src/riscv
brew install riscv-tools
brew edit riscv-gnu-toolchain
```

To install 32-bit cross-compiler, update the variables `args` to use `rv32i` architecture (32-bit integer) and `ilp32` (soft floating points)

```
args = [
    "--prefix=#{prefix}",
    "--with-cmodel=medany",
    "--with-arch=rv32i",
    "--with-abi=ilp32"
]
```

Then build the toolchain from source

```
brew reinstall --build-from-source riscv-gnu-toolchain
```

✓ Useful Utilities / Docking areas / Reminder for all

The following are useful commands or tools for the projects.

Serial port connection

On a Rocky Linux terminal (EIETL/DPO), run the command

```
screen /dev/ttyUSB2 115200
```

On a Mac terminal, run the command

```
screen /dev/tty.usbserial-ib9cplCK1 115200
```

Reference Books/Links

Available across different colleges

[Digital design and computer architecture \(RISC-V\)/ Sarah L. Harris, David Harris.](#)